

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 25

Duration :60 Mins
On Class
Total Marks:50

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

SHORT ANSWER TEST

Q1. A lexical analyzer produces the following token stream:

id + id

Explain how the parser validates the syntactic structure of the expression.

Q2. Given the grammar:

$S \rightarrow aS \mid b$

Determine whether the string aaab belongs to the language using leftmost derivation.

Q3. Remove left recursion from the grammar:

$A \rightarrow Aa \mid b$

Q4. Find FIRST(S) for the grammar:

$S \rightarrow aA \mid b$

$A \rightarrow c \mid \epsilon$

Q5. Determine whether the grammar is suitable for top-down parsing.

$S \rightarrow Sa \mid b$

Justify your answer.

Q6. Apply left factoring to the grammar:

$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

Q7. Construct a parse tree for the string:

a + b

using the grammar:

$E \rightarrow E + E$

$E \rightarrow \text{id}$

Q8. Write the recursive descent procedures for the grammar:

$S \rightarrow aA$

$A \rightarrow b$

Q9. Given the grammar:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

Construct the predictive parsing table.

Q10. Demonstrate the first three actions of a shift-reduce parser for the input:

id + id

Q11. Identify the handle in the string:

id + id

for the grammar:

$E \rightarrow E + E$

$E \rightarrow \text{id}$

Q12. Construct the operator precedence relations for the operators:

+

*

where * has higher precedence than +.

Q13. Explain how operator precedence parsing handles the expression:
id * id

Q14. What is an LR parser? Illustrate the parsing process using the grammar:
 $S \rightarrow aA$
 $A \rightarrow b$

Q15. Given the grammar:
 $S \rightarrow aA$
 $A \rightarrow b$
Write the steps involved in constructing an SLR parsing table.

Q16. Compute FOLLOW(A) for the grammar:
 $S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow b$

Q17. Construct the augmented grammar for:
 $S \rightarrow a$

Q18. Find the closure of the LR(0) item:
 $S' \rightarrow \bullet S$
for the grammar:
 $S \rightarrow a$

Q19. Check whether the grammar is ambiguous:
 $E \rightarrow E + E$
 $E \rightarrow id$
using the string:
id + id + id

Q20. Construct a leftmost derivation for the string:
id + id
using the grammar:
 $E \rightarrow E + E$
 $E \rightarrow id$

Q21. Apply left factoring to the grammar:
 $S \rightarrow aA$
 $S \rightarrow aB$
 $A \rightarrow b$
 $B \rightarrow c$

Q22. Find FIRST(A) for the grammar:
 $A \rightarrow x$
 $A \rightarrow y$

Q23. Determine whether the grammar is suitable for predictive parsing.
 $S \rightarrow aA \mid aB$
 $A \rightarrow b$
 $B \rightarrow c$
Justify your answer.

Q24. Construct recursive descent parser procedures for the grammar:

$S \rightarrow aS$

$S \rightarrow b$

Q25. Build the predictive parsing table for the grammar:

$S \rightarrow aS$

$S \rightarrow b$

Show the table entries for the terminals:

a, b, \$

COURSE CODE : CSA1408	NAME: R. Ranjitha
COURSE NAME: Compiler Design:-	REG. NO: 192421285
	DATE: 06/08/2026
Short Answer Test:-	
① Token Stream :- The lexical analyzer produces the tokens $id + id$. The parser checks whether the token sequence matches the grammar (eg:- $E \rightarrow E + E / id$) $id \rightarrow$ Valid Expression $+ \rightarrow$ operator $id \rightarrow$ valid expression Hence, $id + id$ is syntactically valid.	
② Leftmost derivation: Given :- $S \rightarrow aS/b$ $S \rightarrow ab \quad S \rightarrow aaS \quad S \rightarrow b$ $S \rightarrow aaab$ Therefore, $aaab$ belongs to the language:-	
③ Left recursion:- Given:- $A \rightarrow Aa/b$ $A \rightarrow bA' \quad A' \rightarrow aA'/\epsilon$	

④ FIRST (S)

Grammar :-

$$S \rightarrow aA/b$$

$$A \rightarrow c/\epsilon$$

$$\text{FIRST}(S) = \{a, b\}$$

⑤ Top-down parsing:-

Grammar : $S \rightarrow Sa/b$

Not suitable for Top-down parsing because it contains left recursive.

⑥ Left Factoring:-

Given:-

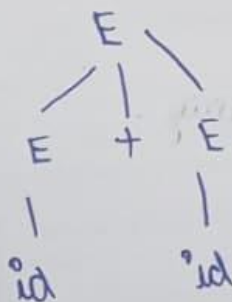
$$S \rightarrow \text{if } E \text{ then } S$$

$$S \rightarrow \text{if } E \text{ then } S \text{ else } S$$

$$S \rightarrow \text{if } E \text{ then } SS'$$

$$S' \rightarrow \text{else } S/\epsilon$$

⑦ parse Tree:-



⑧ Recursive descent procedure:-

S()

```

{
    AC()
}

AC()
{
    if (input == 'b')
    {
        input ++ ;
        AC()
    }
    return ;
}

main()
{
    S()
    if (input == '$')
        printf ("parsing success");
}

```

⑨ predictive parsing table:

Non terminal	a	b	\$
S	$S \rightarrow AB$		
A	$A \rightarrow a$		
B		$B \rightarrow b$	

⑩ Shift - reduce parser:

Stack	Input Buffer	Action:
\$ E	id + id \$	$E \rightarrow id$ Shift
\$ E id	+ id \$	Reducer $id \rightarrow E$
\$ E	+ id \$	Shift

⑪ handle

Grammar $E \rightarrow E + E$, $E \rightarrow id$

For $id + id$, the first handle is id .

② operator precedence parsing :-

since $*$ has higher precedence than $+$;

$\bullet * > +$

$\bullet + < *$

③ operator precedence parsing handles :-

Stack	Input buffer	Action
\$	$id + id \$$	Shift id
$\$ id$	$+ id \$$	Shift $+$
$\$ id +$	$id \$$	Shift id
$\$ id + id$	$\$$	-

Reduce $id + id$ to an expression because $*$ has higher precedence.

④ LR parser :-

A bottom up - parser that scans input left to right and produces the Rightmost derivation in reverse.

Parsing :-

$\$ \rightarrow$ Shift a

$\$ a \rightarrow$ Shift b

$\$ ab \rightarrow$ reduce $A \rightarrow b$

$\$ a \rightarrow$ reduce $S \rightarrow aA \quad \therefore$ Accept

⑮ SLR parsing table:-

Steps of SLR

Augment the grammar

Construct LR (0) items

Compute closure and goto

Compute FIRST and FOLLOW steps

Fill ACTION and GOTO tables

check for conflicts.

⑯ FOLLOW (A)

A is followed by B

FOLLOW (A) = FIRST (B) = $\{b\}$

⑰ Augmented grammar:-

$S \rightarrow a$

$S' \rightarrow S$

$S \rightarrow a$

⑱ closure of LR (0) item:-

$S \rightarrow a$

$S' \rightarrow \cdot S$

$S' \rightarrow \cdot S$

$S \rightarrow a$

⑲ Ambiguous:-

$E \rightarrow E + E$

$E \rightarrow id$

string :- $id + id + id$

Ambiguous because two different parse trees

• $(id + id) + id$

• $id + (id + id)$

②①. leftmost derivation:-

$$\begin{aligned} E &\rightarrow E + E & E &\rightarrow \underline{E} + E \\ E &\rightarrow id & &= id + E \\ & & &= id + \underline{id} \end{aligned}$$

②② leftmost factoring:-

$$\begin{aligned} S &\rightarrow aA & S &\rightarrow aS' \\ S &\rightarrow aB & S' &\rightarrow A/B \\ A &\rightarrow b & A &\rightarrow b \\ B &\rightarrow c & B &\rightarrow c \end{aligned}$$

②③ FIRST (A)

$$\begin{aligned} A &\rightarrow x \\ A &\rightarrow y \end{aligned} \quad \text{FIRST (A)} = \{x, y\}$$

②④ predictive parsing:-

$$\begin{aligned} S &\rightarrow aA / aB & \text{Not suitable because both} \\ A &\rightarrow b & \text{Production begin with a. Causing} \\ B &\rightarrow c & \text{a FIRST / FIRST conflict.} \end{aligned}$$

②⑤ recursive descent parser:

```

SC()
{
    if (input == 'a')
    {
        input++;
    }
}

```

```

    SC()
    {
        if (input == 'b')
        {
            input++;
        }
    }
}

```

```

main()
{
    SC()
    if (input == '\0')
        printf("parsing success")
}

```

②⑥ predictive parsing

Table:-

$$\begin{aligned} S &\rightarrow aS \\ S &\rightarrow b \end{aligned}$$

Non Terminal	a	b	\$
S	S → aS	S → b	

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand